

Faculty of Information Technology (FIT) – Ho Chi Minh City University of Science (HCMUS)

CS423 / CSC13003 – Software Testing (AI-augmented · 2026)

AI POLICY · TEMPLATES — 2026 v1.0

AI Audit Report — 5-section Template per Artifact

Mandatory appendix for every AI-assisted homework (HW#01–HW#06, and Seminar).

*Adapted from Med Kharbach, PhD (2026) — AI Use Policy Templates for Higher Education. CC BY-NC-SA 4.0.
This adaptation is prepared for FIT@HCMUS – CS423 / CSC15003 Software Testing course.*

1. Student Information

Field	Value
Student name (printed):	Cao Trần Bá Đạt
Student ID:	23127168
Class / Cohort:	23KTPM1
Assignment ID (e.g., HW#00, HW#02):	HW#02
Assignment date:	10/06/2026
AI tool(s) used:	Gemini
AI tool(s) used:	☒ Yes [] No

2. Instructions (read before filling)

- Add one row per AI-generated artifact (test case, script, checklist, OpenAPI spec, JMeter plan, etc.).
- Paste the verbatim prompt — DO NOT paraphrase.
- Paste the verbatim AI output (or include a labelled screenshot in the report).
- Tag the verdict: VALID / INVALID / INCOMPLETE.
- Reasoning must cite a course slide, ISTQB section, or technical RFC.
- Show the corrected artifact with the change highlighted.
- Sample rows are in italic — replace them before submission.

3. Audit Table — one row per artifact

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
Tool: Gemini Time: 12:07 10/06/2026 Input: README.md (from Eshop	I have carefully reviewed both documents and am fully prepped to step into the role of a professional tester to help you tackle this project. Here is a brief breakdown of	VALID	AI has summarized the basic features described in the Eshop README.md file and the steps for performing domain	None.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
repo), S04_Domain Testing. Prompt: Imagine you are a professional tester. Here are my two document files including the md. file describing the Eshop system and remaining file is a document about domain testing. You must understand all the concepts in these files before I move on to the next step.	my understanding to confirm we are on the same page: 1. EShop System Specifications (System Requirements) I understand the EShop platform consists of four main components (Backend, Web, Admin, Mobile) and has strict business rules. Key areas of focus for testing will likely include: • <i>Authentication:</i> Complex password rules, a 3-strike login lockout, and a 2-step OTP password reset process. • <i>Checkout & Coupons:</i> Backend recalculation of totals and a strict 5-condition validation for applying discount codes. • <i>Order State Machine:</i> A defined flow from pending to delivered or canceled, with specific constraints on who can cancel orders at the shipping stage. • <i>Admin Functions:</i> JWT and role-based access control, along with an all-or-nothing atomic rollback requirement for CSV product imports. 2. Domain Testing Framework I am ready to apply the 4-step domain testing approach to the EShop requirements: • <i>Step 1:</i> Identifying input and output variables from the SRS. • <i>Step 2:</i> Defining valid and invalid Equivalence Classes (partitions) to avoid redundant testing.		testing in the S04_Domain_Testing.pdf file.	

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	• <i>Step 3:</i> Selecting the best representative test cases from each partition. • <i>Step 4:</i> Applying Boundary Value Analysis (BVA) to catch edge-case errors, testing specifically at the boundaries (e.g., LB, LB+1, UB-1, UB). I am ready when you are. Which EShop module or specific functional requirement (FR) would you like us to design test cases for first?			
Tool: Gemini Time: 12:20 10/06/2026 Prompt: Now, I want you help me step 1 that you must understand each condition in the FR-02 description and then with each condition fields above, you must identify equivalence classes based on guildlines in SO4_Domain Testing.pdf (focused on step 1 and step 2 in this file)	Step 1: Identify Input & Output Variables Inputs: • Email Format: The string entered into the email field. • Credentials Match: The combination of the entered email and password checked against the database records. • Failed Attempts Counter (X): The current number of consecutive incorrect login attempts prior to this login action. Outputs: • Authentication Status: Success (returns a JWT Token) or Failure (returns an appropriate error message). • Account State: Active or Temporarily locked for 30 seconds. • Counter State: Incremented by exactly 1 unit (on failure) or reset. Step 2: Identify Equivalence	INCOMPLETE	Artificial intelligence has created the classes as described in README.md, but it is still missing a class for the condition that the user enters an email and password.	I asked the AI to add a user login and password condition class to the user input condition.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	Classes According to the domain testing guidelines, we partition the domain into sub-domains (equivalence classes). 1. Condition: Email Format <i>Rule: The email field must use HTML5 type="email" validation.</i> • EC1 (Valid): Has valid email format Passes HTML5 validation, proceeds to backend check. • EC2 (Invalid): Does not have a valid email format Blocked by UI validation. 2. Condition: Credentials Match <i>Rule: User must input correct Email and Password. Error messages must not reveal detailed causes.</i> • EC3 (Valid): Email exists AND Password is correct Login success, returns JWT Token. • EC4 (Invalid): Email exists BUT Password is incorrect Login fails, generic error, increments counter. • EC5 (Invalid): Email does not exist in the system Login fails, generic error. 3. Condition: Consecutive Failed Login Attempts (X) <i>Rule: If incorrect logins reach 3 or more consecutive times, the account is temporarily locked for 30 seconds.</i> • EC6 (Valid): $0 \leq X < 3$ Allows normal login attempt.			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	• EC7 (Invalid): $X \geq 3$ Rejects login, returns locked account error (30s). • EC8 (Invalid): $X < 0$ Impossible state (Counter cannot be negative). 4. Output Equivalence Classes • EC9: Authentication Status Success: Returns JWT Token • EC10: Authentication Status Failure: Returns generic error message without exposing the exact cause • EC11: Account State Temporarily Locked for 30 seconds			
Tool: Gemini Time: 15:44 10/06/2026 Prompt: After reviewing FR-02 in the README.md file, I noticed you're missing an equivalence class for the condition that the user has entered a username and password. Please review this condition in FR-02 in the README and add the equivalence class for this requirement.	You are absolutely right to catch that! I appreciate your sharp eye. In my previous breakdown, I jumped straight to the format and matching of the credentials but skipped the most fundamental "must be" condition: the user actually providing the inputs in the first place. According to FR-02, the very first requirement is "Người dùng nhập Email và Mật khẩu" (User inputs Email and Password). Based on the Domain Testing guidelines, when an input condition specifies a "must be" situation, we need to identify one valid equivalence class and one invalid equivalence class. Because there are two distinct fields here, leaving	VALID	The AI has added the exact missing layer that I requested.	None.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	either one blank constitutes a separate invalid state that the program might handle differently. Here is the newly added equivalence class table to complete Step 2 for FR-02: 0. Condition: Presence of Input Data <i>Rule: The user must enter both Email and Password before submitting the login form.</i> <i>Guideline Applied: "Must be" condition. Splitting the invalid classes...</i> • EC_NEW_1 (Valid): Email is NOT empty AND Password is NOT empty Form is allowed to submit; proceeds to Format Validation (EC1). • EC_NEW_2 (Invalid): Email is empty (null/blank) UI prevents submission; prompts user to enter Email. • EC_NEW_3 (Invalid): Password is empty (null/blank) UI prevents submission; prompts user to enter Password. • EC_NEW_4 (Invalid): Both Email AND Password are empty UI prevents submission; prompts user to fill required fields. Adding this ensures our test coverage starts from the absolute baseline of user interaction. Excellent catch!			
Tool: Gemini Time: 15:56	Designing test cases is actually Step 3 in the domain testing	INCOMPLETE	The test cases generated by artificial	I asked the AI to redo

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
10/06/2026 Prompt: Next, in step 2, I want you to design test cases to cover the classes you mentioned above (the test cases must actually be based on the Domain Testing documentation I provided). You must adhere to the following principles in my documentation: - Choose at least one test case from each equivalence class. - For valid classes, choose test cases to cover as many equivalence classes as possible, until all valid classes have been covered. - For invalid classes, choose test cases so that each covers one and only one invalid class, until all classes are covered.	framework, but let's dive right in and apply those strict selection guidelines to the equivalence classes we just established. To build the minimum set of test cases, we will strictly follow the two golden rules from the documentation: • Maximize Valid Coverage: We will create a test case that bundles as many valid equivalence classes as possible into a single scenario. • Isolate Invalid Classes: We will create separate test cases for each invalid equivalence class, ensuring that every invalid test case contains only one invalid condition while keeping all other inputs valid. Here is the test case suite for FR-02 (Đăng nhập), mapped to the "Minimum Set of Test Cases" structure. <i>(Note: For the test data, I am utilizing the test@eshop.com / Test1234! credentials provided in the EShop System Specifications).</i> Minimum Set of Test Cases for FR-02 • TC1: Partitions: EC_NEW_1, EC1, EC3, EC6 ($0 \leq X < 3$), EC9 Inputs: test@eshop.com, Test1234!, 0 Pass: Login successful, system returns JWT Token. • TC2: Partitions: EC_NEW_2 Inputs: (empty), Test1234!, 0 Fail: UI prevents submission,		intelligence are complete and cover all the equivalent classes created above; however, they still lack the objectives and test steps for each test case. Test case 9 cannot be executed because it requires modification of the source code.	the goal setting and testing steps to make the content more complete. I will remove test case TC9.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	prompts for Email. • TC3: Partitions: EC_NEW_3 Inputs: test@eshop.com, (empty), 0 Fail: UI prevents submission, prompts for Password. • TC4: Partitions: EC_NEW_4 Inputs: (empty), (empty), 0 Fail: UI prevents submission, prompts for required fields. • TC5: Partitions: EC2 Inputs: user_at_domain.com, Test1234!, 0 Fail: HTML5 form validation blocks submission. • TC6: Partitions: EC4, EC10 Inputs: test@eshop.com, WrongPass1!, 0 Fail: Generic error displayed. Failed attempts counter increments to 1. • TC7: Partitions: EC5, EC10 Inputs: nobody@eshop.com, Test1234!, 0 Fail: Generic error displayed. Failed attempts counter increments to 1. • TC8: Partitions: EC7 (X >= 3), EC11 Inputs: test@eshop.com, Test1234!, 3 Fail: Login rejected, returns "account temporarily locked for 30s" error. • TC9: Partitions: EC8 (X < 0) Inputs: test@eshop.com, Test1234!, -1 Fail: Backend validation error (Impossible state handled gracefully). Why this structure works: • TC1 does the heavy lifting by proving the "happy path" works when all conditions are valid.			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	• TC2 through TC9 are precision strikes. By isolating the invalid conditions (e.g., in TC5, the password and counter are completely valid, only the email format is wrong), we ensure that if the test fails, we know exactly which business logic module in the code caused the failure.			
Tool: Gemini Time: 17:38 10/06/2026 Prompt: After reviewing the test cases you provided, I realized that both objective and test steps are necessary, so please create these two sections for each test case.	Here is the detailed test suite for FR-02: Đăng nhập & Khóa tài khoản, with clear objectives and step-by-step execution instructions for each case. I have used the default test account (test@eshop.com / Test1234!) and the default frontend URL (http://localhost:5173) provided in the system specifications. TC1: Successful Login with Valid Credentials • Partitions Tested: EC_NEW_1 (Inputs not empty), EC1 (Valid email format), EC3 (Credentials match), EC6 (Failed attempts $0 \leq X < 3$), EC9 (Success output). • Objective: Verify that a user can log in successfully, receive a JWT token, and access the system when providing valid, matching credentials on an active account. • Test Steps: 1. Open a browser and navigate to the EShop frontend (http://localhost:5173). 2. Ensure the backend failed attempts counter for this	VALID	The objectives and test steps are all created in accordance with their corresponding test cases.	None.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	account is currently less than 3. 3. Enter test@eshop.com into the Email field. 4. Enter Test1234! into the Password field. 5. Click the login submission button. • Expected Result: The system authenticates the user, returns a JWT Token to be saved on the client side, and grants access. TC2: Login Attempt with Empty Email Field • Partitions Tested: EC_NEW_2 (Email field empty). • Objective: Verify that the system blocks form submission and prompts the user when the Email field is missing data. • Test Steps: 1. Navigate to the EShop frontend login screen (http://localhost:5173). 2. Leave the Email field blank. 3. Enter Test1234! into the Password field. 4. Attempt to submit the form. • Expected Result: The UI prevents form submission and displays a validation error indicating the email is required. TC3: Login Attempt with Empty Password Field • Partitions Tested: EC_NEW_3 (Password field empty). • Objective: Verify that the system blocks form submission and prompts the user when the Password field is missing data.			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	• Test Steps: 1. Navigate to the EShop frontend login screen. 2. Enter test@eshop.com into the Email field. 3. Leave the Password field blank. 4. Attempt to submit the form. • Expected Result: The UI prevents form submission and displays a validation error indicating the password is required. TC4: Login Attempt with Both Fields Empty • Partitions Tested: EC_NEW_4 (Both Email and Password fields empty). • Objective: Verify that the system completely blocks form submission when no credentials are provided. • Test Steps: 1. Navigate to the EShop frontend login screen. 2. Leave both the Email and Password fields blank. 3. Attempt to submit the form. • Expected Result: The UI prevents submission and displays validation alerts for both required fields. TC5: Login Attempt with Invalid Email Format • Partitions Tested: EC2 (Invalid email format). • Objective: Verify that the HTML5 type="email" validation blocks submission for malformed email strings. • Test Steps: 1. Navigate to the EShop frontend login screen. 2. Enter an invalid string (e.g.,			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	user_at_domain.com) into the Email field. 3. Enter Test1234! into the Password field. 4. Attempt to submit the form. • Expected Result: The browser's native HTML5 validation intercepts the submission, highlighting the email field as improperly formatted. TC6: Login Attempt with Incorrect Password • Partitions Tested: EC4 (Wrong password), EC10 (Generic Error). • Objective: Verify that an incorrect password results in a generic failure message and strictly increments the failed attempts counter by 1. • Test Steps: 1. Ensure the user account test@eshop.com has a failed attempts counter of 0. 2. Navigate to the EShop frontend login screen. 3. Enter test@eshop.com into the Email field. 4. Enter WrongPassword123! into the Password field. 5. Submit the form. • Expected Result: Login fails. The system displays a generic error message (not revealing whether the password was the specific cause) and the backend counter increments to 1. TC7: Login Attempt with Non-Existent Email • Partitions Tested: EC5 (Email does not exist), EC10 (Generic Error).			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	• Objective: Verify that entering an unregistered email returns the exact same generic error message to prevent account enumeration. • Test Steps: 1. Navigate to the EShop frontend login screen. 2. Enter unregistered_user@eshop.com into the Email field. 3. Enter Test1234! into the Password field. 4. Submit the form. • Expected Result: Login fails. The UI displays the exact same generic error message as seen in TC6. TC8: Account Locked Due to Excessive Failures (X >= 3) • Partitions Tested: EC7 (Failed attempts X >= 3), EC11 (Account locked 30s). • Objective: Verify that the system rejects valid logins and enforces a 30-second temporary lockout after 3 consecutive failed attempts. • Test Steps: 1. Intentionally fail the login for test@eshop.com 3 consecutive times to push the counter to 3. 2. Immediately attempt to log in using the correct credentials (test@eshop.com / Test1234!). 3. Submit the form. • Expected Result: The system rejects the login despite correct credentials and returns an error indicating the account is temporarily locked for 30 seconds. TC9: Boundary Anomaly —			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	Negative Counter State ($X < 0$) • Partitions Tested: EC8 (Failed attempts $X < 0$). • Objective: Verify that an impossible negative counter state is handled securely by the backend and does not grant unauthorized access. • Test Steps: 1. Via database manipulation or API interception, set the consecutive failed login attempts counter to -1 for test@eshop.com. 2. Navigate to the EShop frontend login screen. 3. Enter test@eshop.com and Test1234!. 4. Submit the form. • Expected Result: The backend handles the boundary anomaly gracefully (either by resetting the counter to 0 and proceeding, or throwing a backend validation error) without compromising system security.			
Tool: Gemini Time: 11:37 10/06/2026 Prompt: Based on the documentation in S04_Domain testing, follow the instructions therein and perform Boundary Value Analysis. Test cases must include Test ID information,	The final step in the Domain Testing technique is Boundary Value Analysis (BVA). According to the provided documentation, the program is more likely to fail at a boundary due to mis-specified inequalities (e.g., using $\leq$ instead of $<$) or mistyped boundary values. Based on requirement FR-02 of the EShop system, we have two variables containing boundary values that require analysis:	INCOMPLETE	Most of the test cases in Boundary Value Analysis are correct. However, the TC_BVA_04 test case requires code modification, which is invalid.	Delete test case TC_BVA_04.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
Objective, Precondition (if any), Test steps, and Expected Result.	• Consecutive failed login attempts (X): The condition dictates that the account is locked if the user fails 3 or more times. The valid region is $0 \leq X \leq 2$. The invalid (locked) region is $X \geq 3$. • Account lockout time (T): The condition states the account is temporarily locked for exactly 30 seconds. The locked region is $0 \leq T < 30$. The unlocked region is $T \geq 30$. Below is the Test Case suite applying the BVA strategy, focusing specifically on Lower Boundary (LB), Lower Boundary + 1 (LB+1), Upper Boundary - 1 (UB-1), Upper Boundary (UB), and Upper Boundary + 1 (UB+1). Set 1: BVA for Consecutive Failed Login Attempts (X) • TC_BVA_01: Test at the Lower Boundary (LB) of the valid region ($X = 0$) <i>Objective:</i> Ensure the system allows a normal login when the failed attempts counter is at its smallest possible value. <i>Precondition:</i> The account test@eshop.com currently has 0 consecutive failed login attempts. <i>Test steps:</i> 1. Navigate to the login page. 2. Enter the Email test@eshop.com and the correct Password. 3. Click "Đăng nhập" (Login). <i>Expected Result:</i> Login is successful, and the system			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	returns a JWT Token. • TC_BVA_02: Test at the Upper Boundary (UB) of the valid region (X = 2) <i>Objective:</i> Ensure that when the user has failed 2 times (at the upper edge of the valid inputs), the system still allows them to make a 3rd attempt. <i>Precondition:</i> The account test@eshop.com currently has 2 consecutive failed login attempts. <i>Test steps:</i> 1. Navigate to the login page. 2. Enter the Email test@eshop.com and the correct Password. 3. Click "Đăng nhập". <i>Expected Result:</i> Login is successful; the system resets the failed attempt counter to 0 and returns a JWT Token. • TC_BVA_03: Crossing from UB to UB+1 / Lower Boundary of the invalid region (X = 3) <i>Objective:</i> Catch "Inequalities mis-specified" errors (e.g., if a developer mistakenly wrote $X > 3$ instead of $X \geq 3$). Ensure that on exactly the 3rd failed attempt, the account gets locked. <i>Precondition:</i> The account currently has 2 failed attempts. <i>Test steps:</i> 1. Navigate to the login page. 2. Enter the Email test@eshop.com and an incorrect Password. 3. Click "Đăng nhập" (incrementing the backend counter from 2 to 3). <i>Expected Result:</i> Login fails. The system immediately locks			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	the account for 30 seconds and displays an appropriate error message. • TC_BVA_04: Test at LB+ 1 of the invalid region (X = 4) <i>Objective:</i> Ensure that after the account is already locked (past the threshold of 3), attempting to bypass restrictions via API calls that further increment the counter will still be blocked. <i>Precondition:</i> By manipulating the database or API, set the failed attempts for test@eshop.com to 4. The account is in a locked state. <i>Test steps:</i> 1. Navigate to the login page. 2. Enter the Email test@eshop.com and the correct Password. 3. Click "Đăng nhập". <i>Expected Result:</i> Login fails because the account is still correctly recognized as locked. Set 2: BVA for Account Lockout Time (T) • TC_BVA_05: Test at UB-1 of the lockout time (T = 29 seconds) <i>Objective:</i> Test just before the unlock threshold to ensure the system does not unlock the account faster than intended. <i>Precondition:</i> The account test@eshop.com was just locked due to 3 failed attempts. A stopwatch is ready. <i>Test steps:</i> 1. Wait exactly 29 seconds from the exact moment of lockout. 2. Enter the correct Password and click "Đăng nhập".			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	<i>Expected Result:</i> The system rejects the login and notifies the user that the account is still temporarily locked. • TC_BVA_06: Test at UB of lockout time / LB of unlock time (T = 30 seconds) <i>Objective:</i> Ensure the system unlocks exactly at the 30-second boundary as specified to catch off-by-one errors (e.g., $T > 30$ instead of $T \geq 30$). <i>Precondition:</i> The account test@eshop.com was just locked. <i>Test steps:</i> 1. Wait exactly 30 seconds from the exact moment of lockout. 2. Enter the correct Password and click "Đăng nhập". <i>Expected Result:</i> Login is successful; the system unlocks the account, resets the failure counter, and returns a JWT Token. • TC_BVA_07: Test at LB+1 of the unlock time (T = 31 seconds) <i>Objective:</i> Ensure the account maintains normal operational status after safely passing the unlock threshold. <i>Precondition:</i> The account test@eshop.com was just locked. <i>Test steps:</i> 1. Wait 31 seconds from the exact moment of lockout. 2. Enter the correct Password and click "Đăng nhập". <i>Expected Result:</i> Login is successful, yielding the exact same outcome as TC_BVA_06.			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
Tool: Gemini Time: 1:01 11/06/2026 Prompt: Based on the steps above, please help me re-implement the equivalence classes section for the FR-09 description in the README.md file.	Here is the detailed implementation specifically for Step 2 (Identify Equivalence Classes) regarding requirement FR-09: Discount Code (Coupon), strictly adhering to the equivalence partitioning principles in the Domain Testing guidelines. According to the EShop system requirements, a discount code is only applied if it simultaneously satisfies all 5 conditions (C1 to C5) and uses one of 2 different calculation formulas depending on the code type. Based on the principle of "splitting the equivalence class into two or more smaller equivalence classes if there is any reason to believe that elements in an equivalence class are not handled in an identical manner by the program", we partition the valid and invalid data domains as follows: Equivalence Classes Analysis for FR-09 • EC1 (Valid): C1: Existence & Status Code exists in DB AND is_active = 1 → Passes condition C1. • EC2 (Invalid): C1: Existence & Status Code does NOT exist in DB → Error: Code does not exist. • EC3 (Invalid): C1: Existence & Status Code exists in DB BUT is_active = 0 → Error: Code has been disabled.	VALID	Equivalent classes that adequately meet the descriptions in section FR-09 of the VALID and INVALID classes.	None.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	• EC4 (Valid): C2: Expiration Current time < expired_at → Passes condition C2. • EC5 (Invalid): C2: Expiration Current time >= expired_at → Error: Code has expired. • EC6 (Valid): C3: Order Threshold Total order amount >= min_order_amount → Passes condition C3. • EC7 (Invalid): C3: Order Threshold Total order amount < min_order_amount → Error: Minimum order amount not reached. • EC8 (Valid): C4: User Authentication Request includes a valid JWT Token → Passes condition C4. • EC9 (Invalid): C4: User Authentication Request has no Token or an invalid Token → Error: Login required. • EC10 (Valid): C5: Usage Limit User's previous uses < max_uses_per_user → Passes condition C5. • EC11 (Invalid): C5: Usage Limit User's previous uses >= max_uses_per_user → Error: Usage limit reached for this code. • EC12 (Valid): Discount Type type attribute = 'percent' → System applies formula: total * discount_value / 100. • EC13 (Valid): Discount Type type attribute = 'fixed' → System applies formula: discount_value. • EC14 (Invalid): Discount Type Invalid type attribute (e.g., empty string) → Error: Invalid code configuration			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	data. Rationale for Analysis: • Conditions C2, C3, and C5 specify a range of values. According to the guidelines, we establish 1 Valid region and 1 Invalid region for each of these inequality ranges. • Conditions C1 and C4 represent "must be" situations. Specifically for C1, since there are 2 distinct error states at the database level (code not found vs. found but inactive), we split it into EC2 and EC3 to accurately catch developers' underlying logic errors. • The Type attribute represents a set of values handled by different code branches (formulas) in the system. Therefore, we create separate Valid classes for both 'percent' and 'fixed' to ensure test cases cover both algorithmic branches.			
Tool: Gemini Time: 1:15 11/06/2026 Prompt: Next, you should design test cases (excluding BVA) as you did before. Remember to include a Test ID, Objective, Precondition, Input, Test Steps, and Expected Result	Here is the detailed test case design for FR-09: Discount Code (Coupon), strictly applying Step 3 of the Domain Testing guidelines. According to the documentation, we must maximize the coverage of valid equivalence classes in a single test, while ensuring that each invalid test case covers one and only one invalid condition to isolate errors. <i>(Note: These test cases utilize the sample discount codes</i>	INCOMPLETE	The AI test cases mostly meet the FR-09 description but still lack cases for calculating the final_amount after applying the coupon.	I requested the AI to generate additional test cases for calculating the final_amount for fixed and percent coupon types.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
for each test case.	<i>provided in the system specifications).</i> 1. Valid Test Cases (Maximizing Coverage) • TC_FR09_01: Partitions Tested: EC1 (Active), EC4 (Not Expired), EC6 (Meets Min Order), EC8 (Authenticated), EC10 (Under Usage Limit), EC12 (Type: Percent) Objective: Verify that a valid "percent" type discount code is applied and calculated accurately. Precondition: User test@eshop.com is logged in. 0 previous uses of SAVE10. Input: Cart Total = 300,000 ₺, Coupon = SAVE10 Steps: 1. Navigate to Checkout. 2. Verify total. 3. Enter SAVE10. 4. Click Apply. Expected Result: System accepts code. A 30,000 ₺ (10%) discount is applied. final_amount is 270,000 ₺. • TC_FR09_02: Partitions Tested: EC1, EC4, EC6, EC8, EC10, EC13 (Type: Fixed) Objective: Verify that a valid "fixed" type discount code is applied and calculated accurately. Precondition: User test@eshop.com is logged in. 0 previous uses of BIGBUY. Input: Cart Total = 500,000 ₺, Coupon = BIGBUY Steps: 1. Navigate to Checkout. 2. Verify total. 3. Enter BIGBUY. 4. Click Apply. Expected Result: System accepts code. A 50,000 ₺ fixed discount is applied. final_amount is 450,000 ₺.			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	2. Invalid Test Cases (Isolating Errors) • TC_FR09_03: Partitions Tested: EC2 (Code does not exist) Precondition: User is logged in. Input: Cart Total = 500,000 ₺, Coupon = INVALID99 Steps: 1. Go to Checkout. 2. Enter INVALID99. 3. Click Apply. Expected Result: System rejects code, displays error: "Code does not exist." • TC_FR09_04: Partitions Tested: EC3 (Code exists but inactive) Precondition: User logged in. DB has DISABLED50 with is_active = 0. Input: Cart Total = 500,000 ₺, Coupon = DISABLED50 Steps: 1. Go to Checkout. 2. Enter DISABLED50. 3. Click Apply. Expected Result: System rejects code, displays error indicating code is no longer active. • TC_FR09_05: Partitions Tested: EC5 (Code expired) Precondition: User logged in. Code has expired_at in the past. Input: Cart Total = 500,000 ₺, Coupon = EXPIRED Steps: 1. Go to Checkout. 2. Enter EXPIRED. 3. Click Apply. Expected Result: System rejects code, displays error indicating code has expired. • TC_FR09_06: Partitions Tested: EC7 (Minimum order not met) Precondition: User logged in. BIGBUY requires			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	500,000 ₪. Input: Cart Total = 490,000 ₪, Coupon = BIGBUY Steps: 1. Go to Checkout. 2. Enter BIGBUY. 3. Click Apply. Expected Result: System rejects code, displays error that minimum order amount is not met. • TC_FR09_07: Partitions Tested: EC9 (Unauthenticated / No JWT) Precondition: User is browsing as a guest. Input: Cart Total = 500,000 ₪, Coupon = BIGBUY Steps: 1. Go to Checkout. 2. Enter BIGBUY. 3. Click Apply. Expected Result: System prevents application and prompts user to log in. • TC_FR09_08: Partitions Tested: EC11 (Usage limit reached) Precondition: User logged in. User already applied BIGBUY max allowed times. Input: Cart Total = 500,000 ₪, Coupon = BIGBUY Steps: 1. Go to Checkout. 2. Enter BIGBUY. 3. Click Apply. Expected Result: System rejects code, displays error indicating usage limit reached.			
Tool: Gemini Time: 1:28 10/062026 Prompt: After reviewing, I noticed you haven't included test cases for checking the	You are absolutely right. While the previous valid test cases touched on the final amount, creating dedicated test cases that explicitly verify the mathematical logic for both percent and fixed discount formulas ensures the backend recalculation is rigorously tested.	VALID	The test cases were created exactly as I requested.	None.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
final_amount for the two coupon types. Therefore, please create test cases for this scenario.	According to the specifications, the backend must calculate the discount based on the type and subtract it from the total to get the final_amount. Here are the dedicated test cases for checking the final_amount calculations: Test ID: TC_FR09_CALC_01 (Percent Calculation) • Partitions Tested: EC12 (Type is percent). • Objective: Verify that the system accurately calculates the discount_amount using the percentage formula and correctly subtracts it to yield the final_amount. • Precondition: User is logged in with a valid JWT. The database contains the active code SAVE10 (type: percent, value: 10, min_order: 300,000 ₪). • Input: Cart Total: 850,000 ₪ Coupon Code: SAVE10 • Test Steps: 1. Navigate to the Checkout page. 2. Verify the cart total is exactly 850,000 ₪. 3. Enter SAVE10 into the discount code input field. 4. Click the "Apply" button. 5. Observe the breakdown of the order totals on the UI. • Expected Result: The system calculates the discount amount as: $850,000 * 10 / 100 = 85,000$ ₪. The system calculates the final amount as: $850,000 - 85,000 = 765,000$ ₪.			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	The UI clearly displays the Discount (85,000 đ) and the updated Total (765,000 đ). Test ID: TC_FR09_CALC_02 (Fixed Calculation) • Partitions Tested: EC13 (Type is fixed). • Objective: Verify that the system accurately applies a fixed flat-rate discount_amount and correctly subtracts it to yield the final_amount. • Precondition: User is logged in with a valid JWT. The database contains the active code VIP100 (type: fixed, value: 100,000 đ, min_order: 300,000 đ). • Input: Cart Total: 450,000 đ Coupon Code: VIP100 • Test Steps: 1. Navigate to the Checkout page. 2. Verify the cart total is exactly 450,000 đ. 3. Enter VIP100 into the discount code input field. 4. Click the "Apply" button. 5. Observe the breakdown of the order totals on the UI. • Expected Result: The system identifies the discount amount as the fixed value: 100,000 đ. The system calculates the final amount as: 450,000 - 100,000 = 350,000 đ. The UI clearly displays the Discount (100,000 đ) and the updated Total (350,000 đ).			
Tool: Gemini Time: 1:47 11/06/2026 Prompt:Please	Here is the detailed implementation of Step 4: Boundary Value Analysis (BVA) for the FR-09: Discount Code	INCOMPLETE	The generated test cases meet the requirements described in the README.md file,	I explained to the AI that the expiration

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
perform the next BVA step in the same way as you did previously for this FR-09 section.	(Coupon) requirement. According to the Domain Testing guidelines, BVA focuses on the boundary values of input domains to catch errors like mis-specified inequalities (e.g., using > instead of >=) or off-by-one computational errors. For FR-09, we have three numerical/chronological variables with strict boundaries defined in the specifications: • Minimum Order Amount (min_order_amount): Condition is Total >= min_order_amount. • Usage Limits (max_uses_per_user): Condition is Uses < max_uses_per_user. • Expiration Date (expired_at): Condition is Current Date < expired_at. <i>(Note: The test cases below utilize the sample discount codes provided in the system specifications).</i> Set 1: BVA for Minimum Order Amount (min_order_amount) We will test against the BIGBUY code, which has a strict min_order_amount of 500,000 ₺. • TC_FR09_BVA_01 (LB - 1): Objective: Verify the system rejects the coupon just below the required minimum order threshold. Precondition: User		except for the coupon expiration part, as allowing the system to set a time limit is not feasible.	date cannot be tested using BVA, so I will ask the AI to include these conditions in the non-boundary test case type.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	is logged in and has not used BIGBUY before. Input: Cart Total = 499,999 đ, Coupon = BIGBUY Steps: 1. Navigate to Checkout. 2. Ensure cart total is exactly 499,999 đ. 3. Enter BIGBUY and click "Apply". Expected Result: The system rejects the code because the condition (Total >= 500,000) is strictly not met. An error indicating the minimum order amount is not reached is displayed. • TC_FR09_BVA_02 (LB): Objective: Verify the system accepts the coupon exactly at the minimum order boundary. Precondition: User is logged in and has not used BIGBUY before. Input: Cart Total = 500,000 đ, Coupon = BIGBUY Steps: 1. Navigate to Checkout. 2. Ensure cart total is exactly 500,000 đ. 3. Enter BIGBUY and click "Apply". Expected Result: The system accepts the code, as the >= condition is satisfied, and applies the 50,000 đ discount. • TC_FR09_BVA_03 (LB + 1): Objective: Verify the system safely accepts the coupon just above the minimum order boundary. Precondition: User is logged in and has not used BIGBUY before. Input: Cart Total = 500,001 đ, Coupon = BIGBUY Steps: 1. Navigate to Checkout. 2. Ensure cart total is exactly 500,001 đ. 3. Enter BIGBUY and click "Apply".			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	Expected Result: The system accepts the code and applies the 50,000 đ discount. Set 2: BVA for Usage Limits (max_uses_per_user) We will test against the VIP100 code, which allows a maximum of 2 uses per user (max_uses_per_user = 2). • TC_FR09_BVA_04 (LB of Valid Uses): Objective: Ensure the system allows application when the user has never used the code before. Precondition: User is logged in. The database shows 0 previous uses of VIP100 for this user. Input: Cart Total = 300,000 đ, Coupon = VIP100 Steps: Enter VIP100 at checkout and click "Apply". Expected Result: Code is accepted (Condition: 0 < 2 is True). • TC_FR09_BVA_05 (UB of Valid Uses): Objective: Ensure the system allows application on the absolute final allowed attempt. Precondition: User is logged in. The database shows exactly 1 previous use of VIP100 for this user. Input: Cart Total = 300,000 đ, Coupon = VIP100 Steps: Enter VIP100 at checkout and click "Apply". Expected Result: Code is accepted (Condition: 1 < 2 is True). This consumes their last usage limit. • TC_FR09_BVA_06 (UB + 1 /			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	LB of Invalid Uses): Objective: Catch "mis-specified inequality" errors (e.g., if dev wrote ≤ 2 instead of < 2). Ensure the system blocks the 3rd attempt. Precondition: User is logged in. The database shows exactly 2 previous uses of VIP100 for this user. Input: Cart Total = 300,000 đ, Coupon = VIP100 Steps: Enter VIP100 at checkout and click "Apply". Expected Result: The system rejects the code because the usage limit has been reached (Condition: $2 < 2$ is False). Set 3: BVA for Expiration Date (expired_at) The condition explicitly states the current date must be before expired_at. Let's assume a hypothetical code TESTCODE expires precisely at 2099-12-31 00:00:00. • TC_FR09_BVA_07 (UB - 1 Second): Objective: Verify the code is accepted at the very last valid second before expiration. Precondition: User is logged in. System/Server time is artificially set to 2099-12-30 23:59:59. Input: Coupon Code = TESTCODE Steps: Enter TESTCODE at checkout and click "Apply". Expected Result: The code is accepted. • TC_FR09_BVA_08 (UB / Exact Expiration Boundary): Objective: Verify the system rejects the code at the exact			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	boundary timestamp. "Before" implies an exclusive bound (<), not inclusive (<=). Precondition: User is logged in. System/Server time is artificially set to exactly 2099-12-31 00:00:00. Input: Coupon Code = TESTCODE Steps: Enter TESTCODE at checkout and click "Apply". Expected Result: The code is rejected, displaying an "Expired" error. • TC_FR09_BVA_09 (UB + 1 Second): Objective: Verify the system continues to reject the code after expiration. Precondition: User is logged in. System/Server time is artificially set to 2099-12-31 00:00:01. Input: Coupon Code = TESTCODE Steps: Enter TESTCODE at checkout and click "Apply". Expected Result: The code is rejected, displaying an "Expired" error.			
Tool: Gemini Time: 12:11 11/06/2026 Prompt: After reviewing the coupon expiration date requirement, I realized that this should be in the non-boundary testing section, i.e., in the domain testing section in the	This is a solid approach. Moving the expired_at condition out of Boundary Value Analysis and treating it as a standard Equivalence Partitioning (non-boundary) test makes sense when the execution date is treated as a fixed environment variable rather than an input you can increment by a single second. Using the current testing timeframe of June 2026 (specifically, June 11, 2026), and strictly adhering to the	VALID	The test cases generated by the AI were appropriate and complete with the expiration dates of the coupons described in the README.	None.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
previous BVA. Therefore, please design the expiration date test case as I requested above. The test date should be in June 2026.	existing coupon data provided in the specification, here are the redesigned Equivalence Partitioning test cases for the Expiration Date condition. Non-Boundary Domain Tests for Expiration Date (expired_at) • TC_FR09_EXP_01 (Valid Partition - Code expires in the future): Partitions Tested: EC4 (Current Date < expired_at). All other inputs are valid. Objective: Verify the system successfully accepts a discount code when its predefined expiration date is set in the future relative to the fixed test execution date of June 2026. Precondition: The user is logged in. The database contains the active coupon code SAVE10, which is configured with expired_at = 2099-12-31. The user has 0 previous uses of this code. Input: System Execution Date = June 11, 2026, Cart Total = 300,000 ₺, Coupon = SAVE10 Steps: 1. Navigate to the EShop Checkout page. 2. Enter SAVE10 into the discount code input field. 3. Click the "Apply" button. Expected Result: The system evaluates the condition (June 11, 2026 < Dec 31, 2099) as True. The code is accepted, and a 10% discount is applied to the order. • TC_FR09_EXP_02 (Invalid Partition - Code expired in			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	the past): Partitions Tested: EC5 (Current Date >= expired_at). All other inputs are valid. Objective: Verify the system reliably rejects a discount code when its predefined expiration date has already passed relative to the fixed test execution date of June 2026. Precondition: The user is logged in. The database contains the coupon code EXPIRED, which is configured with expired_at = 2020-01-01. Input: System Execution Date = June 11, 2026, Cart Total = 150,000 đ, Coupon = EXPIRED Steps: 1. Navigate to the EShop Checkout page. 2. Enter EXPIRED into the discount code input field. 3. Click the "Apply" button. Expected Result: The system evaluates the condition (June 11, 2026 < Jan 01, 2020) as False. The system rejects the code and returns an error indicating that the coupon has expired.			
Tool: Gemini Time: 12:17 11/06/2026 Prompt:Similarly to the steps above, please read the FR-17 description in the README.md file to determine the equivalence classes.	Here is the implementation for Step 2 (Identify Equivalence Classes) regarding requirement FR-17: Quản lý Mã Giảm Giá (Coupon CRUD). According to the system specifications for FR-17, an Admin can Add, View, or Delete discount codes. The "Add" and "Update" operations evaluate several mandatory input fields with strict mathematical constraints	VALID	The equivalent classes created have covered the requirements for generating new coupons.	None.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	and data integrity rules. Based on the Domain Testing guidelines to partition ranges, sets, and "must be" situations, here is the equivalence partitioning for the FR-17 input variables: Equivalence Classes Analysis for FR-17 • EC1 (Valid): code (Unique) String is entered AND does not exist in DB Expected Behavior: Passes validation. • EC2 (Invalid): code (Unique) String is entered BUT already exists in DB Expected Behavior: Error: Code must be unique. • EC3 (Invalid): code (Unique) String is empty or null Expected Behavior: Error: Code is required. • EC4 (Valid): type (Enum set) Value is exactly 'percent' Expected Behavior: Passes validation. • EC5 (Valid): type (Enum set) Value is exactly 'fixed' Expected Behavior: Passes validation. • EC6 (Invalid): type (Enum set) Value is NOT 'percent' or 'fixed' (e.g., empty, typo) Expected Behavior: Error: Invalid type. • EC7 (Valid): discount_value (Positive) Numeric value > 0 Expected Behavior: Passes validation. • EC8 (Invalid): discount_value (Positive) Numeric value <= 0 Expected			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	Behavior: Error: Value must be positive. • EC9 (Invalid): discount_value (Positive) Empty or non-numeric input Expected Behavior: Error: Value is required / Invalid format. • EC10 (Valid): expired_at (Mandatory) Valid datetime format Expected Behavior: Passes validation. • EC11 (Invalid): expired_at (Mandatory) Empty, null, or invalid datetime string Expected Behavior: Error: Expiration date is required / Invalid format. • EC12 (Valid): min_order_amount (Range) Numeric value ≥ 0 Expected Behavior: Passes validation. • EC13 (Invalid): min_order_amount (Range) Numeric value < 0 Expected Behavior: Error: Minimum order amount cannot be negative. • EC14 (Invalid): min_order_amount (Range) Empty or non-numeric input Expected Behavior: Error: Amount is required / Invalid format. • EC15 (Valid): max_uses_per_user (Range) Integer value ≥ 1 Expected Behavior: Passes validation. • EC16 (Invalid): max_uses_per_user (Range) Integer value < 1 (e.g., 0, -1) Expected Behavior: Error: Max uses must be at least 1. • EC17 (Invalid): max_uses_per_user (Range)			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	Empty or non-integer input Expected Behavior: Error: Max uses is required / Invalid format. Rationale for Analysis: • Must-Be Conditions: The code and expired_at fields are mandatory "must be" requirements. The code field is further split to distinguish between the UI missing the input (empty) and the database rejecting a duplicate. • Defined Sets: The type field specifies a specific set of allowed inputs (percent, fixed). We create a valid equivalence class for each handled option and an invalid class for anything outside the set. • Numeric Ranges: The variables discount_value (> 0), min_order_amount (>= 0), and max_uses_per_user (>= 1) specify boundaries. Following the guidelines, we establish one valid class for values within the threshold and invalid classes for values falling below the threshold or missing entirely. <i>What specific operation (Add, View, or Delete) would you like to design the test cases or boundary values for next?</i>			
Tool: Gemini Time: 12:47 11/06/2026 Prompt:The next step for FR-17 is to	Here is the detailed test case design for FR-17: Quản lý Mã Giảm Giá (Coupon CRUD), specifically focusing on the "Add/Create" operation.	INCOMPLETE	Most of the properties of a coupon have been covered except for the type of coupon not included in the invalid test cases.	I asked the AI to generate invalid test cases for the

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
design test cases based on equivalence classes as you did before (excluding BVA).	According to Step 3 of the Domain Testing guidelines, we will construct test cases that maximize the coverage of valid equivalence classes in a single scenario, while isolating invalid equivalence classes one by one to pinpoint errors accurately. 1. Valid Test Cases (Maximizing Coverage) • TC_FR17_01 (Create Valid "Percent" Coupon): Partitions Tested: EC1 (Unique Code), EC4 (Type: percent), EC7 (Discount > 0), EC10 (Valid Date), EC12 (Min Order >= 0), EC15 (Max Uses >= 1). Objective: Verify that an Admin can successfully create a new percentage-based discount code when all inputs are valid. Precondition: User is logged in as an Admin (role = 'admin'). The code SUMMER20 does not exist in the database. Input: code: SUMMER20, type: percent, discount_value: 20, expired_at: 2026-12-31 23:59:59, min_order_amount: 0, max_uses_per_user: 1 Test Steps: 1. Navigate to the Coupon Management section in the Web Admin portal. 2. Click "Add New Coupon". 3. Fill in the form with the input data. 4. Click "Save" or "Create". Expected Result: The system successfully creates the coupon and displays it in the Coupon list. A			coupon type.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	success notification is shown to the Admin. • TC_FR17_02 (Create Valid "Fixed" Coupon): Partitions Tested: EC1 (Unique Code), EC5 (Type: fixed), EC7 (Discount > 0), EC10 (Valid Date), EC12 (Min Order >= 0), EC15 (Max Uses >= 1). Objective: Verify that an Admin can successfully create a new fixed-amount discount code when all inputs are valid. Precondition: User is logged in as an Admin. The code WELCOME50K does not exist in the database. Input: code: WELCOME50K, type: fixed, discount_value: 50000, expired_at: 2026-12-31 23:59:59, min_order_amount: 200000, max_uses_per_user: 5 Test Steps: 1. Navigate to the Coupon Management section. 2. Click "Add New Coupon". 3. Fill in the form with the input data. 4. Click "Save". Expected Result: The system successfully creates the fixed-value coupon, adding it to the list with the correct configuration. 2. Invalid Test Cases (Isolating Errors) • TC_FR17_03 (Isolate Duplicate Code): Partitions Tested: EC2 (Code already exists). All other inputs are valid. Objective: Verify the system rejects coupon creation if the Admin enters a code string that is already			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	active or exists in the database. Precondition: User is logged in as an Admin. The code SAVE10 already exists in the system. Input: code: SAVE10 (with all other fields filled validly). Test Steps: 1. Navigate to the Coupon Management section to add a new coupon. 2. Enter SAVE10 in the code field. 3. Fill the remaining fields with valid data. 4. Click "Save". Expected Result: The system prevents creation and returns a validation error stating that the coupon code must be unique. • TC_FR17_04 (Isolate Missing Code): Partitions Tested: EC3 (Code is empty). All other inputs are valid. Objective: Verify the system enforces the mandatory nature of the code field. Precondition: User is logged in as an Admin. Input: code: (left completely blank). Test Steps: 1. Navigate to the add coupon form. 2. Leave the code field empty. 3. Fill the remaining fields with valid data. 4. Click "Save". Expected Result: Form submission is blocked, and a required field validation error is displayed next to the code input. • TC_FR17_05 (Isolate Invalid/Zero Discount Value): Partitions Tested: EC8 (Discount <= 0). All other inputs are valid. Objective:			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	Verify the system rejects a coupon if the discount value is not a positive number. Precondition: User is logged in as an Admin. Input: discount_value: 0 (or a negative number like -10). Test Steps: 1. Navigate to the add coupon form. 2. Enter valid data for all fields except discount_value. 3. Enter 0 in the discount_value field. 4. Click "Save". Expected Result: The system rejects the input and displays an error stating that the discount value must be positive. • TC_FR17_06 (Isolate Negative Minimum Order Amount): Partitions Tested: EC13 (Min Order < 0). All other inputs are valid. Objective: Verify the system enforces the non-negative boundary for the minimum order amount requirement. Precondition: User is logged in as an Admin. Input: min_order_amount: -50000. Test Steps: 1. Navigate to the add coupon form. 2. Enter valid data for all fields except min_order_amount. 3. Enter -50000 into the min_order_amount field. 4. Click "Save". Expected Result: The system rejects the input and displays an error stating that the minimum order amount cannot be less than zero. • TC_FR17_07 (Isolate Invalid			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	Max Uses Per User): Partitions Tested: EC16 (Max Uses < 1). All other inputs are valid. Objective: Verify the system rejects a coupon configuration if the maximum uses allowed per user is less than 1. Precondition: User is logged in as an Admin. Input: max_uses_per_user: 0. Test Steps: 1. Navigate to the add coupon form. 2. Enter valid data for all fields except max_uses_per_user. 3. Enter 0 into the max_uses_per_user field. 4. Click "Save". Expected Result: The system rejects the input and displays an error stating that the usage limit must be at least 1.			
Tool: Gemini Time: 12:57 11/06/2026 Prompt: Does the coupon type include invalid test cases in FR-17? If so, please create them for me.	Yes, the coupon type absolutely requires dedicated invalid test cases! According to the Domain Testing guidelines, when an input condition specifies a predefined set of allowed values (in this case, the enum set percent or fixed from FR-17), we must test the valid options and strictly isolate the invalid ones. Since we previously covered the valid paths (percent and fixed), here is the invalid test case designed to isolate what happens when an Admin attempts to submit a coupon type that is not in the allowed set.	VALID	The generated test case met the invalid test case requirements for this coupon type.	None.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	Invalid Test Case for Coupon Type (type) • TC_FR17_08 (Isolate Invalid Coupon Type): Partitions Tested: EC6 (Value is NOT percent or fixed). All other inputs are valid. Objective: Verify that the system strictly enforces the allowed enum values for the type field and rejects any unsupported input (e.g., if an Admin bypasses the frontend UI dropdown or an API request is manipulated). Precondition: User is logged in as an Admin (role = 'admin'). The code WINTER99 does not exist in the database. Input: code = WINTER99, type = freeshipping (Invalid: Not in the allowed set), discount_value = 20, expired_at = 2026-12-31 23:59:59, min_order_amount = 0, max_uses_per_user = 1 Test Steps: 1. Navigate to the Coupon Management section in the Web Admin portal. 2. Click "Add New Coupon". 3. Fill in the form. (If the UI restricts type via a dropdown, intercept the network request or use an API testing tool like Postman to manually inject the payload with "type": "freeshipping"). 4. Submit the creation request to the backend. Expected Result: The system safely rejects the payload. A validation error is returned indicating that the type field is invalid and must be either percent or fixed. The			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	coupon is not saved to the database.			
Tool: Gemini Time: 13:25 11/06/2026 Prompt:Next, create BVA test cases for the FR-17 requirement using the same method as before.	Here is the detailed implementation of Step 4: Boundary Value Analysis (BVA) for the FR-17: Quản lý Mã Giảm Giá (Coupon CRUD) requirement. According to the FR-17 specifications, when an Admin creates or updates a discount code, there are three specific numeric fields with explicit boundaries: • discount_value: Must be a positive number (> 0). • min_order_amount: Must be greater than or equal to zero (>= 0). • max_uses_per_user: Must be greater than or equal to one (>= 1). <i>(Note: Assuming the base currency is VND, the smallest valid logical increment for money and uses is 1).</i> Set 1: BVA for Discount Value (discount_value > 0) • TC_FR17_BVA_01 (LB / Invalid Boundary): Objective: Verify the system rejects a coupon creation when the discount value is exactly zero, as the requirement strictly states it must be positive. Precondition: User is logged in as an Admin. Code TEST01 does not exist. Input: code: TEST01, discount_value: 0 (All other fields are valid). Test	VALID	The test cases have covered all the boundary value requirements of this function.	None.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	Steps: 1. Navigate to the Add Coupon form. 2. Enter 0 into the discount_value field. 3. Click "Save". Expected Result: The system rejects the input. A validation error indicates that the discount value must be a positive number (greater than 0). • TC_FR17_BVA_02 (LB + 1 / Valid Boundary): Objective: Verify the system accepts a coupon creation at the absolute smallest positive integer value. Precondition: User is logged in as an Admin. Code TEST02 does not exist. Input: code: TEST02, discount_value: 1 (All other fields are valid). Test Steps: 1. Navigate to the Add Coupon form. 2. Enter 1 into the discount_value field. 3. Click "Save". Expected Result: The system successfully creates the coupon with a discount value of 1. Set 2: BVA for Minimum Order Amount (min_order_amount >= 0) • TC_FR17_BVA_03 (LB - 1 / Invalid Boundary): Objective: Verify the system rejects a minimum order amount just below the allowed threshold. Precondition: User is logged in as an Admin. Code TEST03 does not exist. Input: code: TEST03, min_order_amount: -1 (All other fields are valid). Test Steps: 1. Navigate to the			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	Add Coupon form. 2. Enter -1 into the min_order_amount field. 3. Click "Save". Expected Result: The system rejects the input. A validation error indicates that the minimum order amount cannot be less than 0. • TC_FR17_BVA_04 (LB / Exact Valid Boundary): Objective: Verify the system safely accepts a coupon requiring no minimum order amount (exactly 0). Precondition: User is logged in as an Admin. Code TEST04 does not exist. Input: code: TEST04, min_order_amount: 0 (All other fields are valid). Test Steps: 1. Navigate to the Add Coupon form. 2. Enter 0 into the min_order_amount field. 3. Click "Save". Expected Result: The system successfully creates the coupon. • TC_FR17_BVA_05 (LB + 1 / Valid Boundary): Objective: Verify the system accepts a minimum order amount just above the boundary. Precondition: User is logged in as an Admin. Code TEST05 does not exist. Input: code: TEST05, min_order_amount: 1 (All other fields are valid). Test Steps: 1. Navigate to the Add Coupon form. 2. Enter 1 into the min_order_amount field. 3. Click "Save". Expected Result: The system successfully creates the coupon.			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	Set 3: BVA for Maximum Uses Per User (max_uses_per_user >= 1) • TC_FR17_BVA_06 (LB - 1 / Invalid Boundary): Objective: Verify the system rejects a usage limit set to zero, catching errors if developers mis-specified the inequality (e.g., >= 0 instead of >= 1). Precondition: User is logged in as an Admin. Code TEST06 does not exist. Input: code: TEST06, max_uses_per_user: 0 (All other fields are valid). Test Steps: 1. Navigate to the Add Coupon form. 2. Enter 0 into the max_uses_per_user field. 3. Click "Save". Expected Result: The system rejects the input. A validation error indicates the usage limit must be at least 1. • TC_FR17_BVA_07 (LB / Exact Valid Boundary): Objective: Verify the system accepts a usage limit of exactly 1. Precondition: User is logged in as an Admin. Code TEST07 does not exist. Input: code: TEST07, max_uses_per_user: 1 (All other fields are valid). Test Steps: 1. Navigate to the Add Coupon form. 2. Enter 1 into the max_uses_per_user field. 3. Click "Save". Expected Result: The system successfully creates the coupon with a single-use restriction. • TC_FR17_BVA_08 (LB + 1 /			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	Valid Boundary): Objective: Verify the system accepts a usage limit just above the boundary. Precondition: User is logged in as an Admin. Code TEST08 does not exist. Input: code: TEST08, max_uses_per_user: 2 (All other fields are valid). Test Steps: 1. Navigate to the Add Coupon form. 2. Enter 2 into the max_uses_per_user field. 3. Click "Save". Expected Result: The system successfully creates the coupon.			
Tool: Gemini Time: 13:38 11/06/2026 Prompt:Please create equivalent classes for me using the same method as above, but for the FR-02 on the mobile app. Do not create test cases with a number of attempts less than 0, as this is not feasible.	Here is the Equivalence Classes (Step 2) analysis for the Login feature (FR-02) on the Mobile App. According to the system specification FR-20: Tính năng Mobile, the mobile application is built with React Native and must implement the full login functionality. The core backend business rules (credentials matching, 3-strike lockout) remain identical to the web version. However, the frontend validation mechanism changes on mobile. Instead of relying on the browser's native HTML5 type="email", a React Native app relies on custom JavaScript regular expressions (Regex) and mobile keyboard types (e.g., keyboardType="email-address").	INCOMPLETE	The equivalent classes have basically been created correctly but are missing one case where both the email and password are blank.	I asked the AI to generate test cases along with this missing equivalence class.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	Per your instructions, the impossible state of negative failed attempts ($X < 0$) has been completely excluded from this domain analysis. Here are the equivalence classes tailored for the Mobile App context, applying the Domain Testing guidelines: Equivalence Classes Analysis for Mobile Login (FR-02 / FR-20) • EC1 (Valid): Input Presence Email is NOT empty AND Password is NOT empty Expected Behavior: Mobile UI allows form submission. • EC2 (Invalid): Input Presence Email is empty (null/blank) Expected Behavior: Mobile UI prevents submission; shows "Email required" validation text. • EC3 (Invalid): Input Presence Password is empty (null/blank) Expected Behavior: Mobile UI prevents submission; shows "Password required" validation text. • EC4 (Valid): Email Format (Mobile UI) String matches standard email regex (e.g., user@domain.com) Expected Behavior: Passes local app validation; proceeds to API call. • EC5 (Invalid): Email Format (Mobile UI) String fails email regex validation Expected Behavior: Mobile UI blocks API call and displays format error. • EC6 (Valid): Credentials			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	Match (Backend) Email exists in DB AND Password is correct Expected Behavior: API returns success. Mobile app saves JWT Token (e.g., using SecureStore or AsyncStorage) and navigates to the Home/Profile screen. • EC7 (Invalid): Credentials Match (Backend) Email exists BUT Password is incorrect Expected Behavior: API returns a generic failure message. Backend counter increments by 1. • EC8 (Invalid): Credentials Match (Backend) Email does not exist in the system Expected Behavior: API returns the exact same generic failure message as EC7 to prevent email enumeration. • EC9 (Valid): Lockout Counter (X) Failed attempts $0 \leq X < 3$ Expected Behavior: System processes the login attempt normally. • EC10 (Invalid): Lockout Counter (X) Failed attempts $X \geq 3$ Expected Behavior: System immediately rejects the attempt and returns a "locked out for 30s" error. Key Rationale for the Mobile Context: • EC4 and EC5: On the web, we test if the browser intercepts the bad email. On mobile, elements in these classes test if the developers successfully implemented custom Regex validation before sending the payload			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	over the network. • EC9 and EC10: The valid range is firmly set at 0 to 2 failed attempts, and the invalid range is 3 or more. Because we removed the negative integer feasibility, the boundary for the lockout counter is perfectly covered by these two classes.			
Tool: Gemini Time: 13:47 11/06/2026 Prompt:Next, you should design test cases for the FR-02 on the mobile app, just as you did previously.	Here is the detailed test case design for the Login feature (FR-02) on the Mobile App, specifically applying Step 3 of the Domain Testing guidelines to maximize coverage for valid classes and isolate invalid classes. <i>(Note: These test cases use the default testing credentials provided in the system specifications: test@eshop.com / Test1234!).</i> 1. Valid Test Case (Maximizing Coverage) • TC_FR02_MOB_01 (Successful Login & Mobile Context): Partitions Tested: EC1 (Inputs Present), EC4 (Valid Email Format), EC6 (Credentials Match), EC9 (Lockout Counter < 3). Objective: Verify that a user can successfully log into the mobile app, the correct keyboard is triggered for the email field, and the JWT token is handled correctly. Precondition: The app is installed on a mobile device/emulator. The user's	INCOMPLETE	Test cases covered the conditions and were similar to the previous web browser test cases in pool A, but these lacked one case is that both email and password are empty.	I asked the AI to generate test cases along with this missing equivalence class.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	account (test@eshop.com) has 0 failed login attempts. Input: Email = test@eshop.com, Password = Test1234! Test Steps: 1. Open the EShop mobile app and navigate to the Login screen. 2. Tap the Email input field and observe the mobile keyboard. 3. Enter test@eshop.com. 4. Tap the Password field, enter Test1234!, and ensure the text is masked. 5. Tap the "Login" button. Expected Result: The OS displays an email-optimized keyboard (containing the @ symbol) when the email field is focused. The backend API authenticates the user successfully. The mobile app securely stores the JWT Token. The app navigates the user to the Home or Profile screen. 2. Invalid Test Cases (Isolating Errors) • TC_FR02_MOB_02 (Isolate Missing Email - Local Validation): Partitions Tested: EC2 (Email is empty). All other inputs are valid. Objective: Verify that the mobile frontend prevents submission and handles the missing email locally without hitting the backend API. Precondition: The user is on the app's Login screen. Input: Email = (Leave blank), Password = Test1234! Test Steps: 1. Leave the Email field empty. 2. Enter Test1234! into the Password field. 3. Tap			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	"Login". Expected Result: The app intercepts the action, does not send a network request, and displays a localized validation error (e.g., "Email is required"). • TC_FR02_MOB_03 (Isolate Missing Password - Local Validation): Partitions Tested: EC3 (Password is empty). All other inputs are valid. Objective: Verify that the mobile frontend requires a password before attempting network authentication. Precondition: The user is on the app's Login screen. Input: Email = test@eshop.com, Password = (Leave blank) Test Steps: 1. Enter test@eshop.com into the Email field. 2. Leave the Password field empty. 3. Tap "Login". Expected Result: The app intercepts the action, prevents the API call, and displays a localized validation error (e.g., "Password is required"). • TC_FR02_MOB_04 (Isolate Invalid Email Format - Local Regex): Partitions Tested: EC5 (Email fails regex). All other inputs are valid. Objective: Verify the React Native app correctly applies Regex to reject malformed emails prior to server submission. Precondition: The user is on the app's Login screen. Input: Email = testeshop.com (Missing the @ symbol),			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	Password = Test1234! Test Steps: 1. Enter testeshop.com into the Email field. 2. Enter the password. 3. Tap "Login". Expected Result: The app intercepts the action and displays an error indicating the email format is invalid. No network request is made. • TC_FR02_MOB_05 (Isolate Incorrect Password - API Validation): Partitions Tested: EC7 (Email exists, Password incorrect). All other inputs are valid. Objective: Verify the backend rejects incorrect credentials and increments the lockout counter. Precondition: The user test@eshop.com has 0 previous failed attempts. Input: Email = test@eshop.com, Password = WrongPassword99! Test Steps: 1. Enter the inputs above. 2. Tap "Login". Expected Result: The API rejects the request. The app displays a generic authentication failure message. The backend failed attempt counter increments to 1. • TC_FR02_MOB_06 (Isolate Non-existent Account - API Validation): Partitions Tested: EC8 (Email does not exist). All other inputs are valid. Objective: Verify the backend rejects unregistered emails using the exact same generic message as a wrong password			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	to prevent email enumeration. Precondition: The email used does not exist in the database. Input: Email = unknown_user@eshop.com, Password = Test1234! Test Steps: 1. Enter the inputs above. 2. Tap "Login". Expected Result: The API rejects the request. The app displays the exact same generic failure message as TC_FR02_MOB_05. • TC_FR02_MOB_07 (Isolate Lockout Threshold): Partitions Tested: EC10 (Lockout Counter >= 3). Objective: Verify the system blocks all login attempts (even with correct credentials) once the 3-strike threshold is met. Precondition: The user test@eshop.com has exactly 3 consecutive failed login attempts on record within the current timeframe. Input: Email = test@eshop.com, Password = Test1234! (Correct password) Test Steps: 1. Enter the inputs above. 2. Tap "Login". Expected Result: The API immediately rejects the request. The app displays an error stating the account is temporarily locked out for 30 seconds.			
Tool: Gemini Time: 13:58 11/06/2026 Prompt:After reviewing it, I noticed you're	You are absolutely right. While standard Equivalence Partitioning often isolates one invalid variable at a time to pinpoint errors, testing the default "all empty" state is a	VALID	The AI generated the missing test case exactly as I requested in the prompt.	None.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
missing a case where both the email and password fields are blank. Please create one for me in both equivalence class và test case	critical real-world UI scenario. When a user simply opens the app and immediately taps "Login," the frontend must be able to handle multiple validation states simultaneously. Here is the addition for both the Equivalence Class and the specific Test Case to cover the scenario where both fields are left blank. 1. New Equivalence Class (Mobile Login) We will add this as a new combination class to your existing table. • EC11 (Invalid): Input Presence (Combination) Email is empty AND Password is empty Expected Behavior: Mobile UI prevents submission; shows validation text for both fields simultaneously. No network request is made. 2. New Test Case • TC_FR02_MOB_08 (Isolate All Missing Inputs - Local Validation): Partitions Tested: EC11 (Both inputs are empty). Objective: Verify that the mobile frontend correctly handles the scenario where a user attempts to log in without providing any credentials, displaying validation errors for all			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	required fields simultaneously without making a backend API call. Precondition: The user has just opened the app and navigated to the Login screen. Both fields are in their pristine, empty state. Input: Email = (Leave completely blank), Password = (Leave completely blank) Test Steps: 1. Open the EShop mobile app and navigate to the Login screen. 2. Do not tap or enter any text into the Email field. 3. Do not tap or enter any text into the Password field. 4. Tap the "Login" button directly. Expected Result: The app successfully intercepts the action. No network request is sent to the backend API. Validation error messages for both fields (e.g., "Email is required" AND "Password is required") are displayed simultaneously next to or below their respective input fields.			
Tool: Gemini Time: 14:03 11/06/2026 Prompt:Next, design the BVA using the same methods as above for the FR-02 for the Mobile App.	Here is the detailed implementation of Step 4: Boundary Value Analysis (BVA) for the FR-02: Login feature on the Mobile App. According to the system requirements, the only numeric variable with a defined boundary in the login process is the consecutive failed login attempt counter, which dictates that an account is locked for 30 seconds upon reaching 3 or more failed	VALID	The test cases have covered the BVA test case requirements.	None.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	attempts. Following your previous instruction, we will completely exclude the mathematically impossible lower bound (less than 0 failed attempts) and focus strictly on the critical upper boundary where the system state shifts from "allowed" to "locked". BVA for Lockout Counter (failed_attempts >= 3) • TC_FR02_MOB_BVA_01 (LB - 1 / Valid Boundary): Objective: Verify the mobile app and backend system allow a login attempt to proceed normally when the user is exactly one attempt below the lockout threshold. Precondition: The user test@eshop.com has exactly 2 consecutive failed login attempts on record in the backend. The app is open on the Login screen. Input: Email = test@eshop.com, Password = Test1234! (Correct password) Test Steps: 1. Enter the email and the correct password. 2. Tap the "Login" button. Expected Result: The system processes the request normally. Because the threshold of 3 was not reached prior to this attempt, the login succeeds, the JWT token is saved, and the backend resets the failed attempt counter to 0.			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	• TC_FR02_MOB_BVA_02 (LB / Exact Boundary): Objective: Verify the system triggers the temporary lockout precisely on the 3rd consecutive failed attempt. Precondition: The user test@eshop.com has exactly 2 consecutive failed login attempts on record. The app is open on the Login screen. Input: Email = test@eshop.com, Password = WrongPassword99! (Incorrect password) Test Steps: 1. Enter the email and the wrong password. 2. Tap the "Login" button. Expected Result: The authentication fails. The backend counter increments to 3, strictly hitting the boundary. The system immediately applies the 30-second lockout and returns the corresponding error message to the mobile UI. • TC_FR02_MOB_BVA_03 (LB + 1 / Invalid Boundary): Objective: Verify the system strictly blocks any login attempts when the counter is at or above the threshold (e.g., simulating a 4th attempt while the account is actively locked). Precondition: The user test@eshop.com has exactly 3 failed attempts on record, and the 30-second lockout window is currently active. Input: Email = test@eshop.com, Password = Test1234! (Correct password) Test Steps: 1. Enter the email and the correct password. 2. Tap the "Login"			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning (ISTQB)	(5) Student Fix
	button before the 30 seconds have expired. Expected Result: The system intercepts the request based on the lockout state. It rejects the login immediately—without evaluating if the newly entered password is correct—and displays the lockout error message.			

4. Summary of AI Accuracy

Metric	Count	Percentage
Total AI-generated artifacts audited	17	
VALID (correct, accepted as-is)	8	47.06%
INVALID (wrong; rejected)	0	0%
INCOMPLETE (acceptable after edits)	9	52.94%

5. Conclusion — When should AI be used (or not)?

Artificial intelligence (AI) really does a good job of reading and understanding assigned documents. Generally, the equivalence classes or test cases it generates largely cover the feature requirements and are appropriate. However, some test cases are invalid (e.g., those requiring code modifications), some don't match the requirements description, and some test cases are missing. I reviewed each test case it generated, manually adjusted the inappropriate ones, selected the ones that matched the requirements description, and instructed the AI to create the features it missed in its test case generation. I repeated this process multiple times across various feature requirements.

6. Mandatory Disclosure

"Equivalence classes, test case design and BVA was initially generated by Gemini; I reviewed and modified FR-02, FR-09, FR-17, added None; bug report and test summary report was written entirely by me. The detailed AI Audit Report is attached as Appendix A. I confirm I did not use AI to generate any artifact listed in the prohibited category."

Signature

Student name (printed):	CAO TRẦN BÁ ĐẠT
Student ID:	23127168
Class / Cohort:	23KTPM1

**Student name
(printed):**

CAO TRẦN BÁ ĐẠT

Course:

CS423 / CSC13003 – Software Testing

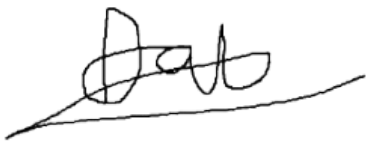
Instructor:

LÂM QUANG VŨ

Date:

10/06/2026

Signature:



Cao Trần Bá Đạt

References

- Kharbach, M. (2026). AI Use Policy Templates for Higher Education. CC BY-NC-SA 4.0.
- ISTQB Foundation Level Syllabus (latest version).
- Hardman, P. (2025). A Post-AI Learning Taxonomy.
- Fuster Rabella, M. (2025). OECD Education Working Paper No. 338.
- Perkins, M., Roe, J., & Furze, L. (2025). AI Assessment Scale.
- Anthropic (2025). Building reliable AI test agents — engineering blog.
- DeepEval & Promptfoo documentation — testing frameworks for LLM systems.